
Travail pratique : Problème du sac à dos binaire

Objectif

L'objectif de ce travail pratique est de programmer un algorithme de programmation dynamique pour la résolution du problème du sac à dos binaire ainsi qu'une heuristique gloutonne pour comparaison.

Problème du sac à dos binaire

On se donne N objets caractérisés chacun une *valeur* entière et positive v_i et un *poids* entier et positif p_i , $i = 1, \dots, N$, ainsi qu'un sac pouvant contenir autant d'objets que voulu tant que leur poids total ne dépasse pas une valeur maximale P .

On cherche la sélection d'objets maximisant la valeur du sac (somme des valeurs des objets sélectionnés) mais ne dépassant pas le poids limite P .

Si on définit les variables de décision

$$x_i = \begin{cases} 1 & \text{si l'objet } i \text{ est sélectionné} \\ 0 & \text{s'il ne l'est pas} \end{cases} \quad i = 1, \dots, N$$

la recherche d'une sélection optimale revient à résoudre le programme linéaire à variables binaires

$$\begin{aligned} \text{Max} \quad & z = \sum_{i=1}^N v_i x_i \\ \text{s.c.} \quad & \sum_{i=1}^N p_i x_i \leq P \\ & x_i \in \{0, 1\}, \quad i = 1, \dots, N \end{aligned}$$

Résolution exacte : Programmation dynamique

Il est possible de résoudre le problème du sac à dos de manière exacte en utilisant les techniques de la programmation dynamique. En une phrase, cette approche consiste à décomposer le problème en une famille de sous-problèmes emboîtés, à établir une relation de récurrence entre les solutions optimales de ces sous-problèmes puis à résoudre cette récurrence en commençant par les sous-problèmes les plus petits.

Dans notre cas, si on note $G_k(y)$ la **valeur optimale d'un sac de capacité y rempli uniquement à l'aide des objets 1 à k** , les valeurs $G_k(y)$ vérifient la relation de récurrence

$$G_k(y) = \begin{cases} G_{k-1}(y) & \text{si } y < p_k \\ \max(G_{k-1}(y), G_{k-1}(y - p_k) + v_k) & \text{si } y \geq p_k \end{cases}$$

pour $k = 1, \dots, N$ et $y = 0, \dots, P$ et avec les valeurs initiales $G_0(y) = 0$ pour $y = 0, \dots, P$. La valeur optimale du problème du sac est alors égale à $G_N(P)$.

L'algorithme présenté en table 1 permet de résoudre la récurrence précédente en place.

TABLE 1 – Algorithme de programmation dynamique pour le problème du sac à dos binaire.

Données : Une liste de $N \geq 1$ objets avec leur valeur (entière et positive) v_k et leur poids entier et positif p_k , $k = 1, \dots, N$, et un poids maximal, entier et positif, P à ne pas dépasser.

Résultat : La valeur maximale d'une sélection d'objets d'un poids total ne dépassant pas P et les tables de décisions optimales.

Début

- (1) Initialiser à zéro un tableau d'entiers F de taille $P + 1$ (indexé de 0 à P)
- (2) Initialiser à zéro un tableau d'entiers/de booléens D de taille $N \times (P + 1)$ (indexé de 1 à N et de 0 à P) // Tables des décisions optimales
- (3) **Pour** k de 1 à N **faire**
- (4) **Pour** y de P à p_k **faire** // L'ordre descendant est essentiel!
- (5) **Si** $F(y) < F(y - p_k) + v_k$ **faire**
- (6) $F(y) \leftarrow F(y - p_k) + v_k$
- (7) $D(k, y) \leftarrow 1$
- (8) **Retourner** la valeur optimale $F(P)$ et les tables D

Fin

REMARQUES.

- a) À l'itération $k = N$ on peut se limiter à considérer la valeur $y = P$.
- b) Les complexités temporelle et spatiale de l'algorithme sont en $O(N \cdot P)$.
- c) Ces complexités sont polynomiales par rapport à N , le nombre d'objets, mais exponentielles par rapport à P , le poids limite du sac (il faut $O(\log_2(P))$ bits pour stocker P).

Résolution approchée : Heuristique gloutonne

Décrire une heuristique gloutonne pour résoudre de manière approchée le problème. Discuter la complexité de votre méthode.

Travail à effectuer

Programmez les deux méthodes de résolution (exacte et approchée) afin de les comparer (valeur des solutions, temps d'exécution) sur quelques jeux de données générés aléatoirement à partir des paramètres suivants :

- ▷ Nombre d'objets : $N = 500$ (valeur par défaut)
- ▷ Poids p_i de l'objet i : entier choisi uniformément entre un poids minimum p_{min} et un poids maximum p_{max} (bornes incluses).
Valeurs par défaut : $p_{min} = 100$ et $p_{max} = 1000$.
- ▷ Valeur v_i de l'objet i : entier égal au poids p_i de l'objet additionné d'une perturbation aléatoire (choisie uniformément) entre $-U$ et U (bornes incluses).
Valeur par défaut : $U = 100$.
- ▷ Poids maximal P du sac : fixé de manière à pouvoir accueillir, en moyenne, une proportion q des objets :

$$P = \lceil q \cdot N \cdot p_{moyen} \rceil = \left\lceil q \cdot N \cdot \frac{p_{min} + p_{max}}{2} \right\rceil$$

Valeur par défaut : $q = 1/4 = 25\%$.